

Arquitectura de ordenadores

6.2 El procesador y ensamblador

Carlos M. Vallez Fernández carlos.vallez@u-tad.com

Nayra Déniz Benítez nayra.deniz@u-tad.com

2025-2026

Índice

1 Intro a Zeus

2 Ensamblador del Z80

1. Intro a Zeus (I)

¿Qué es Zeus?

- Zeus es un ensamblador para el procesador Z80 que fue desarrollado específicamente para el ordenador Sinclair ZX Spectrum.
- Es un ensamblador de dos pasos que permite escribir programas en lenguaje ensamblador Z80 y ensamblarlos para su ejecución en el ZX Spectrum.
- Zeus ofrece una interfaz de usuario fácil de usar que permite editar y ensamblar código en tiempo real, ver y modificar el contenido de la memoria del Spectrum, y ejecutar programas directamente en el emulador de Spectrum incorporado.
- También incluye características avanzadas como la capacidad de importar y exportar programas y datos en varios formatos, y soporte para la depuración de programas con la ayuda de un emulador de depuración incorporado.

1. Intro a Zeus (II)

¿Cómo funciona?

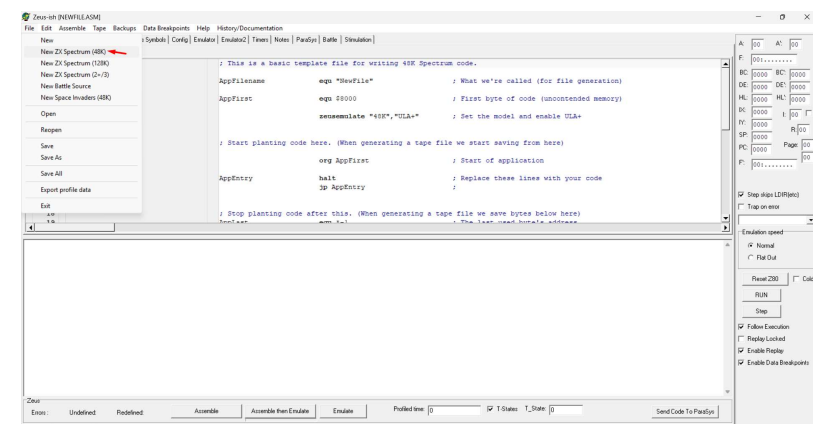
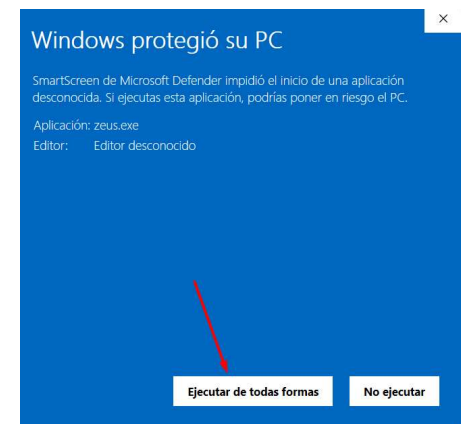
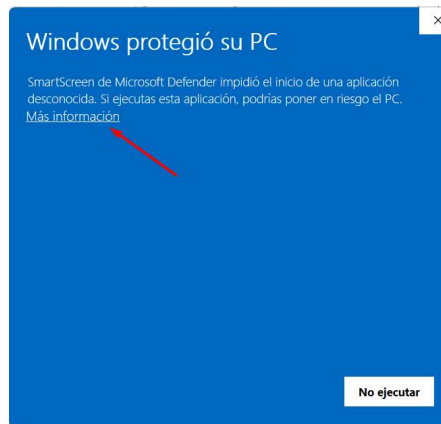
- Cuando ensambla un programa, el ensamblador realiza una primera pasada en la que analiza todo el programa (escrito en ensamblador). En esta pasada:
 - Encuentra todas las etiquetas definidas y crea la tabla de símbolos.
 - Verifica la sintaxis e identifica errores.
- Luego, efectúa una segunda pasada en la que analiza cada instrucción para reconocer:
 - su nemónico y, por lo tanto, la instrucción que se debe ejecutar.
 - los operandos, que pueden ser registros, números o etiquetas. Esto es posible dado que el ensamblador conoce los nombres de los registros, los dígitos, las letras D y H, los símbolos aritméticos y lógicos y los símbolos definidos en la tabla de símbolos.
- El ensamblador ignora todos los campos de comentarios.

1. Intro a Zeus (III)

Instalando para ejecutar



zeus2026.zip



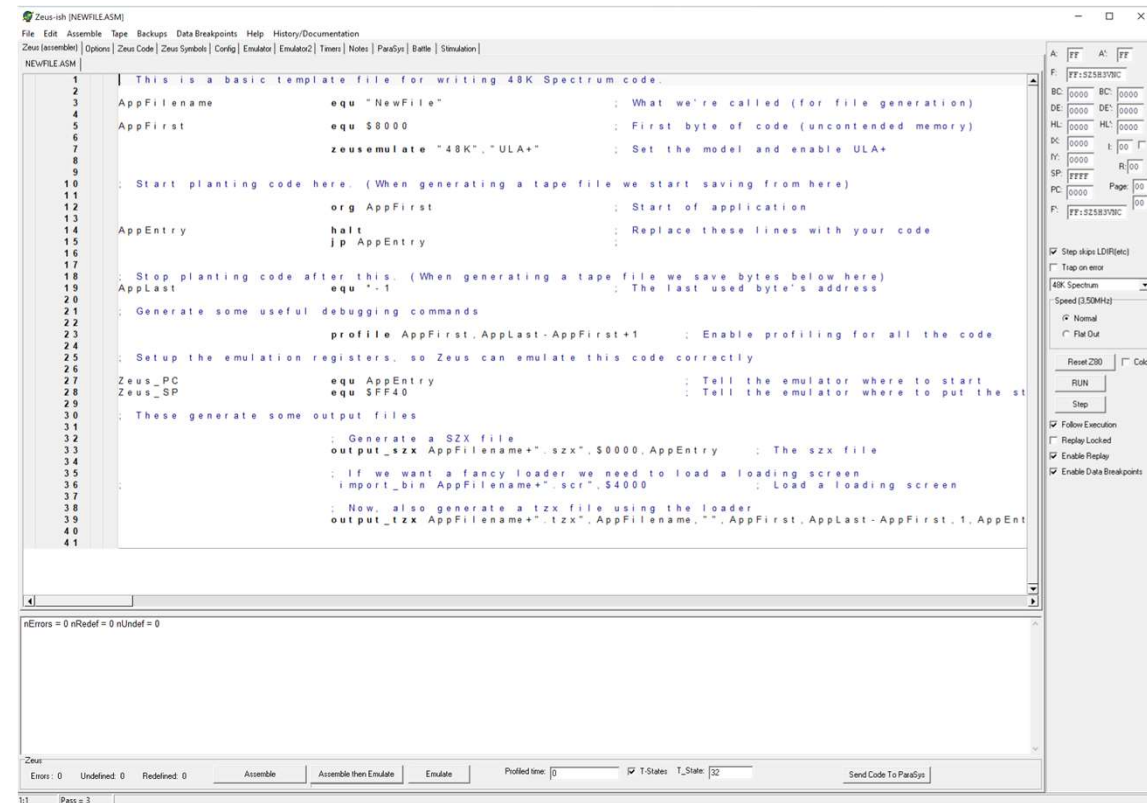
1. Intro a Zeus (IV)

Entorno Zeus

- Look & Feel anticuado – ochentero
- Pestaña Zeus (assembler) es la principal.
- Derecha -> Registros de 8 bits y sus combinaciones. Números en Hexadecimal.
- Botones Rest, Run y Step
- Botón de ensamblado
- Zona Central -> donde escribimos el programa.
- El problema es que el Zeus emula una máquina spectrum -> no podemos poner el código sin más.

Debemos seguir una estructura que el propio ensamblador nos proporciona.

- File -> New ZX Spectrum (48K) -> Esto nos crea un esqueleto vacío.
- Preparado y listo para escribir código. La tipología de letra: en configuración set editor Font Courier 10.



1. Intro a Zeus (IV)

Etiquetas

Many labels: using **EQU**, we assign a value to labels.

Most important labels:

Appfilename: file name

Appfirst: stores the first memory position where we will place our program → the first byte of our program.

The Spectrum's memory is structured in a specific way, so we cannot write just anywhere.

Parameters: to understand the 48K Spectrum version

Between **start** and **halt** is the area where we build our program. It is currently empty.

Below, more parameters (we will not modify that section).

```
; This is a basic template file for writing 48K Spectrum code.

AppFilename      equ "NewFile"                ; What we're called (for file generation)
AppFirst         equ $8000                    ; First byte of code (uncontended memory)
               zeusemulate "48K","ULA+"      ; Set the model and enable ULA+

; Start planting code here. (When generating a tape file we start saving from here)

               org AppFirst                  ; Start of application
AppEntry         halt                        ; Replace these lines with your code
               jp AppEntry                   ;

; Stop planting code after this. (When generating a tape file we save bytes below here)
AppLast         equ *-1                      ; The last used byte's address

; Generate some useful debugging commands

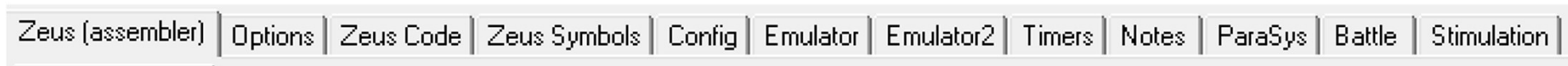
               profile AppFirst,AppLast-AppFirst+1 ; Enable profiling for all the code

; Setup the emulation registers, so Zeus can emulate this code correctly

Zeus_PC         equ AppEntry                 ; Tell the emulator where to start
Zeus_SP         equ $FF40                   ; Tell the emulator where to put the stack
```

1. Intro a Zeus (V)

Pestañas Zeus



- **Zeus(assembler):** Pestaña principal donde escribimos nuestro programa
- **Zeus Code:** Contiene una vista de la memoria.
- **Zeus Symbols:** Contiene las direcciones de los símbolos del sistema.
- **Emulator:** Nos permite emular en caos de tener algo que hacer por pantalla

1. Intro a Zeus (V)

Registros Zeus

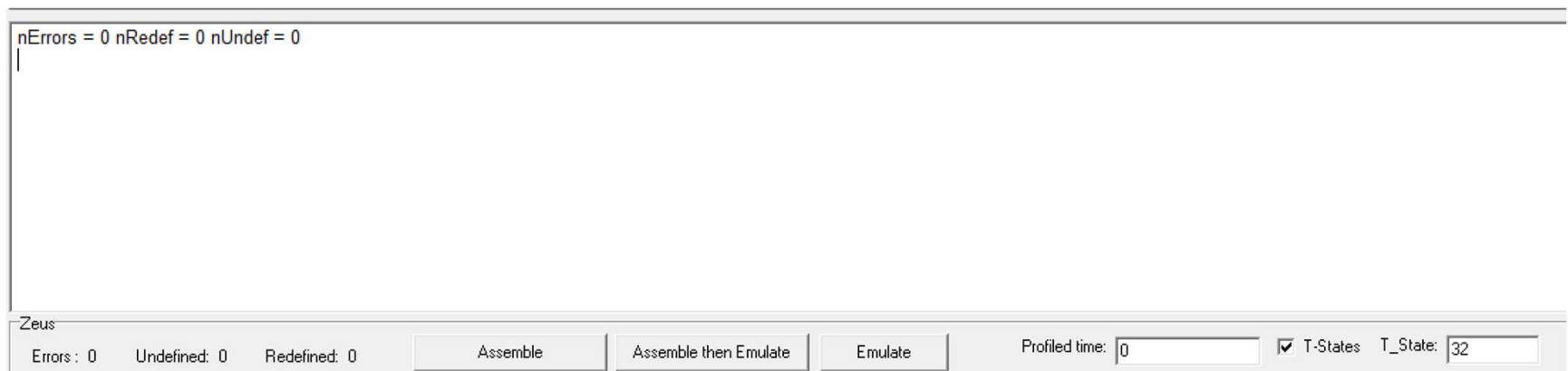
- En la pestaña Zeus 8assembler), a la derecha.
- Podemos ver el valor de todos los registros
- Podemos estar pendientes de:
 - Cambio de valores
 - Los flags
 - El contador de programa
 - ...

A:	FF	A':	FF
F:	FF:SZ5H3VNC		
BC:	0000	BC':	0000
DE:	0000	DE':	0000
HL:	0000	HL':	0000
IX:	0000	I:	00 ☐
IY:	0000	R:	00
SP:	FFFF	Page:	00
PC:	0000		00
F':	FF:SZ5H3VNC		

1. Intro a Zeus (V)

Errores en Zeus

- En la misma pestaña, contamos con una consola que muestra los posibles errores, con un log y el posible origen de los problemas del ensamblador.
- Si el programa no ensambla, mirar aquí la presencia de errores
- Los botones de debajo sirven para ensamblar, emular y ensamblar y emular
- Nosotros usaremos ensamblar (Assemble)
- En los ejercicios veremos errores típicos



1. Intro a Zeus (V)

Memoria en Zeus

- En la pestaña Zeus code, podemos ver una imagen de la memoria.
- Aquí tenemos todo lo que hay en nuestro Spectrum.
- Se organiza según este esquema:

&0000	to	&3FFF	ROM
&4000	to	&57FF	Screen Memory
&5800	to	&5AFF	Screen Memory (Colour Data)
&5B00	to	&5BFF	Printer Buffer
&5C00	to	&5CBF	System Variables
&5CC0	to	&5CCA	Reserved
&5CCB	to	&FF57	Available Memory (between PROG and RAMTOP)
&FF58	to	&FFFF	Reserved

```
000000 F3 AF 11 FF FF C3 CB 11 2A 5D 5C 22 5F 5C 18 43 .....*]\~\_C
000010 C3 F2 15 FF FF FF FF 2A 5D 5C 7E CD 7D 00 D0 .....*]\~.}...
000020 CD 74 00 18 F7 FF FF FF C3 5B 33 FF FF FF FF .....[3.....
000030 C5 2A 61 5C E5 C3 9E 16 F5 E5 2A 78 5C 23 22 78 ..*a\.....*x\#*x
000040 5C 7C B5 20 03 FD 34 40 C5 D5 CD BF 02 D1 C1 E1 \|. ..40.....
000050 F1 FB C9 E1 6E FD 75 00 ED 7B 3D 5C C3 C5 16 FF .....n.u.(=\.....
000060 FF FF FF FF FF FF F5 E5 2A B0 5C 7C B5 20 01 E9 .....*.\|. ..
000070 E1 F1 ED 45 2A 5D 5C 23 22 5D 5C 7E C9 FE 21 D0 ...E*]\#*]\~.!.
000080 FE 0D C8 FE 10 D8 FE 18 3F D8 23 FE 16 38 01 23 .....?..#..8.#
000090 37 22 5D 5C C9 BF 52 4E C4 49 4E 4B 45 59 A4 50 7"]\..RN.INKEY.P
0000A0 C9 46 CE 50 4F 49 4E D4 53 43 52 45 45 4E A4 41 .F.POIN.SCREEN.A
0000B0 54 54 D2 41 D4 54 41 C2 56 41 4C A4 43 4F 44 C5 TT.A.TA.VAL.COD.
0000C0 56 41 CC 4C 45 CE 53 49 CE 43 4F D3 54 41 CE 41 VA.LE.SI.CO.TA.A
0000D0 53 CE 41 43 D3 41 54 CE 4C CE 45 58 D0 49 4E D4 S.AC.AT.L.EX.IN.
0000E0 53 51 D2 53 47 CE 41 42 D3 50 45 45 CB 49 CE 55 SQ.SG.AB.PEE.I.U
0000F0 53 D2 53 54 52 A4 43 48 52 A4 4E 4F D4 42 49 CE S.STR.CHR.NO.BI.
000100 4F D2 41 4E C4 3C BD 3E BD 3C BE 4C 49 4E C5 54 O.AN.<.>.<.LIN.T
000110 48 45 CE 54 CF 53 54 45 D0 44 45 46 20 46 CE 43 HE.T.STE.DEF.F.C
000120 41 D4 46 4F 52 4D 41 D4 D4 4F 56 C5 45 52 41 53 A.FORMA.MOV.ERAS
000130 C5 4F 50 45 4E 20 A3 43 4C 4F 53 45 20 A3 4D 45 .OPEN .CLOSE .ME
000140 52 47 C5 56 45 52 49 46 D9 42 45 45 D0 43 49 52 RG.VERIF.BEE.CIR
000150 43 4C C5 49 4E CB 50 41 50 45 D2 46 4C 41 53 C8 CL.IN.PAPE.FLAS.
000160 42 52 49 47 48 D4 49 4E 56 45 52 53 C5 4F 56 45 BRIGH.INVERS.OVE
000170 D2 4F 55 D4 4C 50 52 49 4E D4 4C 4C 49 53 D4 53 .OU.LPRIN.LLIS.S
000180 54 4F D0 52 45 41 C4 44 41 54 C1 52 45 53 54 4F TO.REA.DAT.RESTO
000190 52 C5 4E 45 D7 42 4F 52 44 45 D2 43 4F 4E 54 49 R.NE.BORDE.CONFI
0001A0 4E 55 C5 44 49 CD 52 45 CD 46 4F D2 47 4F 20 54 NU.DI.RE.FO.GO T
0001B0 CF 47 4F 20 53 55 C2 49 4E 50 55 D4 4C 4F 41 C4 .GO SU.INPU.LOA.
0001C0 4C 49 53 D4 4C 45 D4 50 41 55 53 C5 4E 45 58 D4 LIS.LE.PAUS.NEX.
0001D0 50 4F 4B C5 50 52 49 4E D4 50 4C 4F D4 52 55 CE POK.PRIN.PLO.RU.
0001E0 53 41 56 C5 52 41 4E 44 4F 4D 49 5A C5 49 C6 43 SAV.RANDOMIZ.I.C
0001F0 4C D3 44 52 41 D7 43 4C 45 41 D2 52 45 54 55 52 L.DRA.CLEA.RETUR
000200 CE 43 4F 50 D9 42 48 59 36 35 54 47 56 4E 4A 55 .COP.BHY65TGVNJU
000210 37 34 52 46 43 4D 4B 49 38 33 45 44 58 0E 4C 4F 74RFCMKI83EDX.LO
000220 39 32 57 53 5A 20 D0 50 30 31 51 41 E3 C4 E0 E4 92WSZ .P01QA....
000230 B4 BC BD BB AF B0 B1 C0 A7 A6 BE AD B2 BA E5 A5 .....~.\.{}....
000240 C2 E1 B3 B9 C1 B8 7E DC DA 5C B7 7B 7D D8 BF AE .....~.\.{}....
000250 AA AB DD DE DF 7F B5 D6 7C D5 5D DB B6 D9 5B D7 .....|.}....[.
000260 0C 07 06 04 05 08 0A 0B 09 0F E2 2A 3F CD C8 CC .....*?.....
000270 CB 5E AC 2D 2B 3D 2E 2C 3B 22 C7 3C C3 3E C5 2F .^.-+=,;".<.>./
000280 C9 60 C6 3A D0 CE A8 CA D3 D4 D1 D2 A9 CF 2E 2F .^...../.....
000290 11 FF FF 01 FE FE ED 78 2F E6 1F 28 0E 67 7D 14 .....x/..(g).
0002A0 C0 D6 08 CB 3C 30 FA 53 5F 20 F4 2D CB 00 38 E6 .....<0.S.-..8.
0002B0 7A 3C C8 FE 28 C8 FE 19 C8 7B 5A 57 FE 18 C9 CD 2<...{ZW....
0002C0 8E 02 C0 21 00 5C CB 7E 20 07 23 35 2B 20 02 36 ...!\.~.#5+.6
0002D0 FF 7D 21 04 5C BD 20 EE CD 1B 03 D0 21 00 5C BE .)!.\. ....!\.
0002E0 28 2E EB 21 04 5C BE 28 2B CB 7E 20 04 EB CB 7E (.!\.('~ ~~~
0002F0 C8 5F 77 23 36 05 23 3A 09 5C 77 23 FD 4E 07 FD .w#6.#:.\w#.N..
000300 56 01 E5 CD 33 03 E1 77 32 08 5C FD CB 01 EE C9 V...3..w2.\.....
000310 23 36 05 23 35 C0 3A 0A 5C 77 23 7E 18 EA 42 16 #6.#5:.\w#~.B.
000320 00 7B FE 27 D0 FE 18 20 03 CB 78 C0 21 05 02 19 .{'....x!....
000330 7E 37 C9 7B FE 3A 38 2F 0D FA 4F 03 28 03 C6 4F ~7.{:/8/.O.(.O
000340 C9 21 EB 01 04 28 03 21 05 02 16 00 19 7E C9 21 !....(!.....!
000350 28 02 CB 40 28 F4 CB 5A 28 0A FD CB 30 5E C0 04 ).@((...0^..
```

1. Intro a Zeus (V)

Pestaña Symbols en Zeus

- En la pestaña Zeus Symbols, encontramos las posiciones de memoria donde se guardan los símbolos del sistema (etiquetas).
- Estos estarán en las posiciones que nosotros hayamos detallado en el programa.
- Por ejemplo:
 - AppEntry
 - Inicio
 - Fin
 - ...

Zeus (assembler)	Options	Zeus Code	Zeus Symbols
Allp		EQU \$82C0	
Allp1		EQU \$82CE	
ANDBar		EQU \$85B4	
ANDBase		EQU \$827D	
ANDINV		EQU \$82BE	
ANDInv		EQU \$831E	
ANDIt		EQU \$829D	
ANDShip		EQU \$81FA	
ANDZig		EQU \$8684	
BaseAlive		EQU \$8E2D	
BaseDef1		EQU \$8C5A	
BaseDefn		EQU \$9400	
BaseSpeed		EQU \$8E30	
BaseX		EQU \$8E2E	
BaseY		EQU \$8E2F	
BaseYs		EQU \$89FA	
BitTable		EQU \$883A	

1. Intro a Zeus (V)

Resto de Pestañas en Zeus

NO emplean en esta asignatura



Índice

1 Intro a Zeus

2 Ensamblador Z80

2. Ensamblador Z80

En las siguientes diapositivas nos dedicaremos a aprender diversas instrucciones del ensamblador del procesador Z80

Ensamblador Z80. Estructura Instrucciones

- Las instrucciones en el ensamblador del Z80 tienen la siguiente estructura:



- Donde:
 - Etiqueta**: Un nombre único que identifica de manera unívoca una línea del programa. Es opcional.
 - Operación**: Acoge el nemónico de la operación ensamblador que se desea realizar.
 - Operando(s)**: Corresponde al operando de la operación. Puede ser simple o doble.
 - Si es doble, el primer operando es el destino y el segundo es el origen.
 - Comentarios**: Debe comenzar por el símbolo ';'. Es un espacio para incluir comentarios sobre el programa. Es opcional.

Ensamblador Z80. Ejemplo

- Los **operandos pueden ser registros, direcciones de memoria, valores inmediatos, etiquetas**, etc., dependiendo del tipo de instrucción.
- Algunos ejemplos de instrucciones en ensamblador Z80 son:
 - LD A, 42 ; carga el valor 42 en el acumulador
 - LD B, C ; copia el contenido del registro C en el registro B
 - ADD A, B ; suma el contenido de A y B, almacenando el resultado en A
 - JP 1234h ; salta a la dirección de memoria 1234h
 - CALL mi_subrutina ; llama a la subrutina identificada por la etiqueta "mi_subrutina"
- Ya veremos más adelante con detenimiento cada una de estas instrucciones...

Ensamblador Z80. Estructura Programa

Directiva de comienzo: Indica en qué dirección de memoria está la primera instrucción del programa.

```
program      org 200H
              ld A,37H
              ld (210H),A
              ld C,A
              end program
```

Instrucciones del programa

Etiqueta de inicio: Marca la dirección de memoria del inicio.

Directiva de fin: Indica que el programa se ha acabado.

Ensamblador Z80. Representación.

- **Los números se pueden representar en diferentes bases** en el ensamblador. Algunas instrucciones admiten diferentes bases, otros son dependientes de una base concreta.
 - En **decimal**: 34D o, simplemente, 34. (Ojo 34D no funciona en ZEUS)
 - En **hexadecimal**: 34H o, si comienza por letra, 0A4H
 - En **binario**: 00110100B

```
org 200H

Program  ld A,37H
        ld (210H),A ;source only A
        ld C,A

        end program
```

Ensamblador Z80. LD

- La instrucción que más utilizaremos en nuestros programas en ensamblador será sin duda la **operación de carga o instrucción LD**. Sirve para:
 - Meter un valor en un registro.
 - Copiar el valor de un registro en otro registro.
 - Escribir en memoria (en una dirección determinada) un valor.
 - Escribir en memoria (en una dirección determinada) el contenido de un registro.
 - Asignarle a un registro el contenido de una dirección de memoria.
- La **sintaxis de LD** en lenguaje ensamblador es:

LD DESTINO, ORIGEN

- La instrucción **LD** copia el valor contenido en el operando origen en el operando destino.
- Esta operación no es aritmética, ni lógica ni de desplazamiento, por lo que no afecta a ningún *flag* (F).

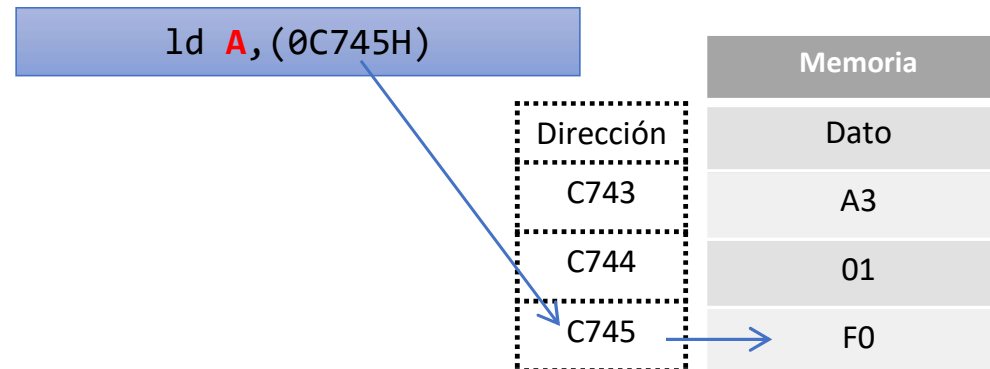
Ensamblador Z80. Direccionamiento

- **Direccionamiento** es la manera en la que se indica a una instrucción dónde están los operandos sobre los que debe operar.
- El ensamblador **soporta diferentes modos de direccionamiento** para acceder a datos y operar con ellos. Los principales modos son:
 - Direccionamiento **extendido**.
 - Direccionamiento **indirecto**.
 - Direccionamiento **indexado**.
 - Direccionamiento **a través de registro**.
 - Direccionamiento **implícito**.
 - Direccionamiento **inmediato**.
 - Direccionamiento **inmediato extendido**.

Ensamblador Z80. Direccionamiento

Direccionamiento extendido

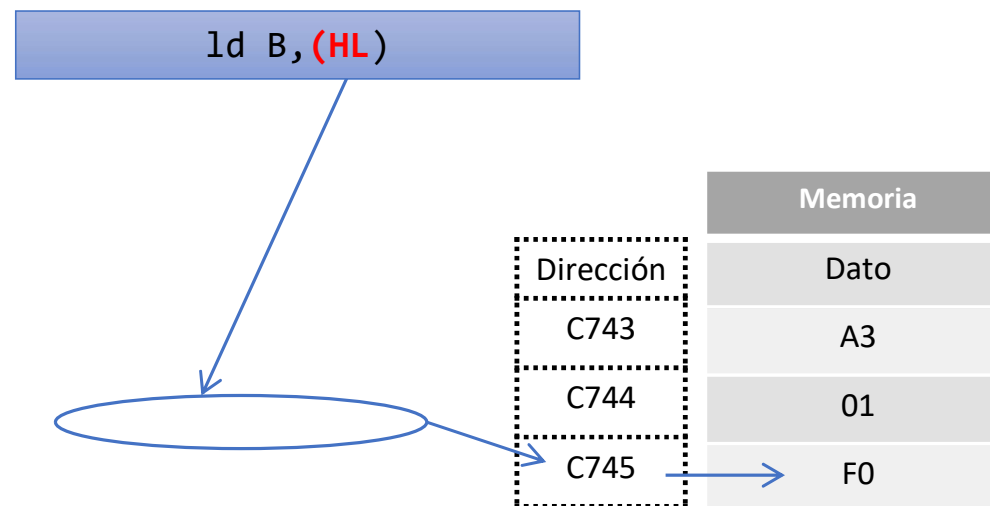
- El **operando es la dirección de memoria** donde está el dato.
 - Las operaciones con memoria solo pueden tener origen o destino A.
 - Si necesitamos cargarlo en otro registro, hay que hacerlo en dos pasos (ej. `ld A, (0C745H)` y después `ld B, A`)
 - El operando tiene 2 bytes, para contener una dirección completa de memoria del Z80.
 - Los registros dobles (BC, DE y HL) también pueden interactuar con la memoria



Ensamblador Z80. Direccionamiento

Direccionamiento indirecto:

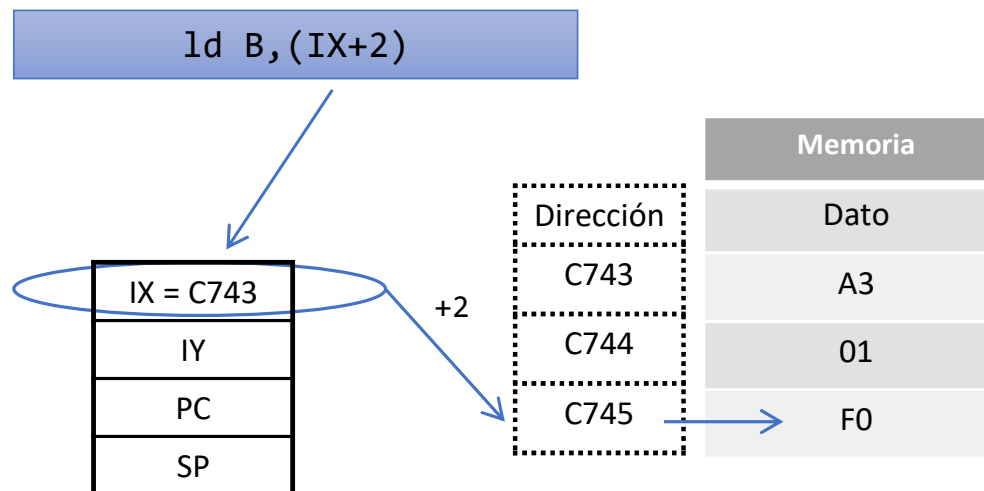
- El **operando es un par de registros** que contienen la dirección de memoria donde está el dato.
 - El operando tiene 2 registros de 1 byte cada uno, para contener una dirección completa de memoria.
 - Todos los registros (A, B, C, D, E, H y L) aceptan como dirección de origen la contenida en el registro (HL). El registro A también acepta, como origen, los registros (BC) y (DE). Todos también aceptan IX e IY



Ensamblador Z80. Direccionamiento

Direccionamiento indexado:

- El **operando es un registro de 2 bytes (IX e IY) más un desplazamiento**.
 - La dirección contenida en el registro más el desplazamiento, resulta en la dirección de memoria donde está el dato. Destino todos los registros de 8 bits (A..L).
 - Todos los registros (A, B, C, D, E, H y L) aceptan como dirección de origen la contenida en los registro IX e IY



Ensamblador Z80. Direccionamiento

Direccionamiento a través de registro:

- **El operando es un registro.**
 - No se accede a memoria, ya que el dato ya está contenido en el registro.
 - Origen y destino todos los registros de 8 bits (A...L)

```
ld B,C
```

Ensamblador Z80. Direccionamiento

Direccionamiento implícito:

- **No hay operandos, la instrucción actúa automáticamente sobre un registro concreto.**
 - No se accede a memoria, ya que el dato ya está contenido en el registro.

rlca

RLCA: rota A un bit hacia la izquierda.

Ensamblador Z80. Direccionamiento

Direccionamiento inmediato:

- El operando es el dato.
 - No se accede a memoria ni a registros.

```
ld B,133D
```

Direccionamiento inmediato extendido:

- Igual que el anterior, pero para números que se van a almacenar en registros de 2 bytes.
 - Registros de 2 bytes; BC, DE, LH, IX, IY..

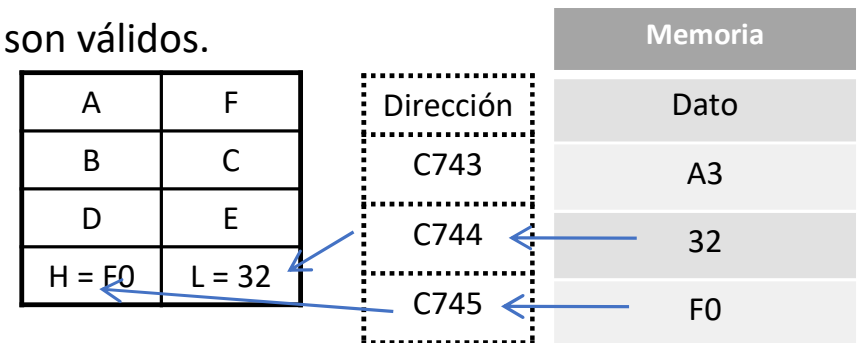
```
ld HL,8000H
```

Ensamblador Z80. LD

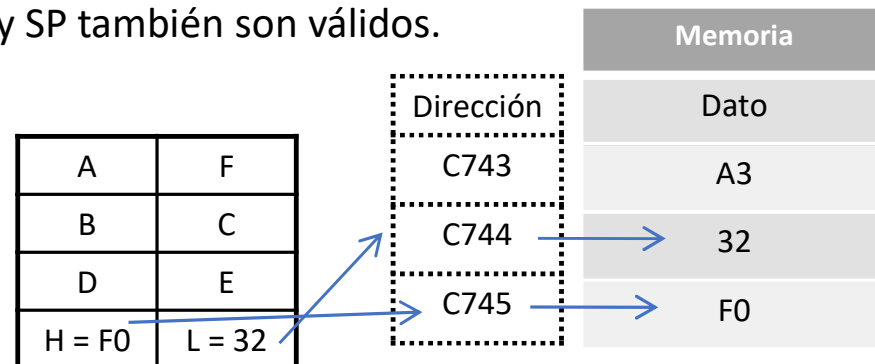
- **Atendiendo a los modos de direccionamiento, la instrucción LD se puede encontrar de las siguientes formas:**
 - LD A, (1000H); sólo registro A
 - LD A, (BC) ; DE y HL también son válidos
 - LD R, (HL) ; donde R = A, B, C, D, E, H, L
 - LD R, (IX + d) ; donde d = desplazamiento
 - LD R, (IY + d); ld HL, (IX+d o IY +d) da error, en cambio con BC y DE no da error
 - LD R, n ; donde n es un número desde 00 a FF y R = A, B, C, D, E, H, L. R puede sustituirse por (HL), (IX + d) e (IY + d)
- Viceversa para **almacenar datos del registro** en lugar de cargar datos en el registro.
 - LD (1000H), A; sólo registro A
- También se puede **transferir el contenido de registro a registro**:
 - LD R1, R2 ; donde R1 = A, B, C, D, E, H, L y R2 = A, B, C, D, E, H, L

Ensamblador Z80. LD

- LD también **admite direccionar datos de 2 bytes**:
 - LD HL, (0C744H); BC, DE, IX, IY y SP también son válidos.



- El byte **menos significativo** está almacenado en la **posición indicada**. El byte **más significativo** en la **posición siguiente**. Esto se llama **Low Endian**.
- Viceversa para almacenar datos de un par de registros en dos direcciones consecutivas de memoria.
 - LD (0C744H), HL ; BC, DE, IX, IY y SP también son válidos.



Ensamblador Z80. LD

- **Transferencia entre registros 2 bytes:**
 - LD SP, HL; IX e IY también son válidos.
- **Carga de números de 2 bytes:**
 - LD BC, nn ; donde nn es un número entre 0000 y FFFF.
 - LD DE, nn
 - LD HL, nn
 - LD SP, nn
 - LD IX, nn
 - LD IY, nn

Ensamblador Z80. Intercambio

- En lugar de cargar o almacenar, es **posible intercambiar valores entre pares de registros con la instrucción EX**:
 - EX DE, HL
 - EX AF, AF'
 - EX (SP), HL
 - EX (SP), IX
 - EX (SP), IY
- Para intercambiar los valores de los registros actuales por los del segundo juego de registros:
 - EXX

Ejercicio 1

- Escribir un programa en ensamblador del Z80 que:
 - Se almacene en la dirección 8000h.
 - Cargue el contenido de la dirección de memoria 300h (5H en mi ejemplo) en el acumulador.
 - Copie el contenido del acumulador en el registro E.
 - Copie el contenido del registro E en el registro B.
 - Marque con la etiqueta “Aquí” cualquiera de las líneas del programa.
 - Que incluya un comentario en la última línea de programa

Solución

org 8000h

ld a, 300h

Aqui ld e, a

ld b, e ; Última línea

Ejercicio 2

Escribir un programa en ensamblador del Z80 que:

- Se almacene en la dirección 8000h.
- Cargue en acumulador (A) el valor 5.
- Cargue en la dirección de memoria 8250h el contenido del acumulador.
- Cargue en acumulador el valor 4.
- Cargue en la dirección de memoria 8251h el contenido del acumulador.
- Cargue en el registro b el contenido del acumulador.
- Sume a y b
- Salvar el resultado en la dirección de memoria 8260h

Solución

```
org 8000h
    ld a, 5
    ld (8250h), a
    ld a, 4
    ld (8251h), a
    ld b, a
    ld a, (8250h)
    add a, b
    ld (8260h), a
    halt
```

Ensamblador Z80. Aritmética y lógica

- La mayoría de las operaciones aritméticas en Z80 se realizan mediante el registro acumulador A.
- Las principales instrucciones aritméticas son:
 - ADD: Suma dos valores y almacena el resultado en un registro.
 - SUB: Resta dos valores y almacena el resultado en un registro.
 - INC: Incrementa el valor de un registro o de una ubicación de memoria en uno.
 - DEC: Decrementa el valor de un registro o de una ubicación de memoria en uno.
- Además, se cuenta con las siguientes instrucciones lógicas:
 - AND: Realiza una operación lógica AND entre dos valores y almacena el resultado en un registro.
 - OR: Realiza una operación lógica OR entre dos valores y almacena el resultado en un registro.
 - XOR: Realiza una operación lógica XOR entre dos valores y almacena el resultado en un registro.
 - NEG y CPL: Hacen complemento a 2 y niegan (respectivamente) lo almacenado en el acumulador.

Ensamblador Z80. Aritmética (ADD)

- ADD se encuentra en las formas:
 - ADD A, R ; donde R = A, B, C, D, E, H, L
 - ADD A, n ; donde n es un número de 00 a FF.
 - ADD A, (HL)
 - ADD A, (IX + d)
 - ADD A, (IY + d)
- ADD también acepta operaciones con registros dobles, de 16 bits. ADD D, S; donde D = HL, IX, IY y S = BC, DE. En este caso el resultado se queda en el primer registro doble, no interviene el registro A. INC y DEC también se pueden aplicar a todos los registros dobles. También admiten las sumas consigo mismo ADD D,D, es una forma de multiplicar por dos, que también se puede obtener con un desplazamiento a la izquierda.
- Si se desea que se sume teniendo en cuenta el acarreo anterior (indicado por el bit C del registro F), la instrucción a usar es ADC

Ensamblador Z80. Aritmética (ADD)

F	7	6	5	4	3	2	1	0
	S	Z	-	H	-	P/V	N	C

- Flags activados por la operación:
 - S = 1 cuando el resultado es negativo.
 - Z = 1 cuando el resultado es cero.
 - H = 1 cuando hay acarreo en el bit 3.
 - P/V = 1 cuando hay overflow.
 - C = 1 cuando hay acarreo en el bit 7.

Ensamblador Z80. Ejemplo ADD

; este programa suma dos números y muestra el resultado en pantalla

LD A, 10 ; carga el primer número en A

LD B, 20 ; carga el segundo número en B

ADD A, B ; suma A y B, almacenando el resultado en A

LD C, A ; copia el resultado en el registro C

CALL mostrar_resultado ; llama a una subrutina (faltaría implementarla)

; fin del programa

Ejercicio 3

Escribir un programa en ensamblador del Z80 que:

- Almacene los números 44H y 2AH en las direcciones de memoria 250H y 251H.
- Almacene la suma de ambos números en la dirección 260H.

Solución

ld a, 44h

ld (250h), a

ld a, 2ah

ld (251h), a

ld b, a

ld a, (250h)

add a, b

ld (260h), a

Ensamblador Z80. Aritmética (SUB)

- SUB realiza una operación de resta entre dos operandos y almacena el resultado en un registro especificado.
 - Es análoga a la operación de suma (ADD), pero el primer operando siempre es el acumulador A, por ello la sintaxis no se incluye.
 - Al contrario de ADD, sólo admite operaciones con registros de 8 bits.
 - También se puede hacer resta con Carry con SBC
- Flags activados por la operación:
 - S = 1 cuando el resultado es negativo.
 - Z = 1 cuando el resultado es cero.
 - H = 1 cuando hay acarreo en el bit 3.
 - P/V = 1 cuando hay overflow.
 - N (Subtract Flag): este bit se establece en 1 debido al uso de la instrucción SUB, y se borra en caso contrario.
 - C = 1 cuando hay acarreo en el bit 7.

Ejercicio 4

- Programar un programa que realice los siguientes pasos:
 - Carga el valor 10 en el acumulador A.
 - Carga el valor 5 en el registro B.
 - Resta el valor en B al valor en A mediante la instrucción SUB.

Solución

org 8000h ; Dirección de inicio del programa

; Carga del primer número

ld a, 10 ; Carga el valor 10 en el acumulador A

; Carga del segundo número

ld b, 5 ; Carga el valor 5 en el registro B

; Resta de los números

sub b ; Resta el valor en B al valor en A

ret ; Finaliza el programa

Ensamblador Z80. Aritmética (INC y DEC)

- INC Incrementa en 1 el valor referido por el operando. Sólo se aplica sobre 8 bits, se puede usar para incrementar valores almacenados en posiciones de memoria
 - INC R ; donde R = A, B, C, D, E, H, L
 - INC (HL)
 - INC (BC)
 - INC (DE)
 - INC (IX + d)
 - INC (IY + d)
- La función DEC es análoga, pero decrementa en 1 el valor referido por el operando.
- Los flags S, Z y H se activan como en ADD.
- P/V se activa si el valor en R antes de la operación es 7FH (INC) o 80H (DEC).
- C no se ve afectado.

Ensamblador Z80. Lógica (AND)

- En ensamblador, la instrucción AND se utiliza para realizar una operación lógica AND a nivel de bit entre el acumulador y otro operando. El resultado se almacena en el acumulador y se actualizan los flags del registro F.
- La sintaxis general de la instrucción AND en ensamblador Z80 es la siguiente:

AND operando

- Donde operando puede ser un registro, una dirección de memoria o un valor inmediato.
- Después de ejecutar la instrucción AND, los flags del registro F se actualizan de la siguiente manera:
 - El bit de signo (S) se pone a cero.
 - El bit de cero (Z) se pone a uno si el resultado es cero; de lo contrario, se pone a cero.
 - El bit de paridad/imparidad (P/V) se pone a uno si el resultado tiene un número par de bits activos; de lo contrario, se pone a cero.
 - El bit de sobrecarga/medio acarreo (H) se pone a uno si la operación ha generado un acarreo a nivel de bit 3-4; de lo contrario, se pone a cero.
 - El bit de acarreo (C) se pone a cero.

Ensamblador Z80. Lógica (AND)

- Registro: se especifica un registro de 8 bits como operando. Ejemplo: AND A.
- Inmediato: se especifica un valor de 8 bits como operando. Ejemplo: AND 0x0F.
- Directo: se especifica una dirección de memoria como operando. El valor se obtiene de la dirección de memoria especificada. Ejemplo: AND (0x1234).
- Indirecto: se especifica un registro que contiene una dirección de memoria como operando. El valor se obtiene de la dirección de memoria apuntada por el registro. Ejemplo: AND (HL) o AND (IX+d).
- Cada modo de direccionamiento tiene su propia sintaxis y su propio propósito. Por ejemplo, el modo inmediato se utiliza para realizar operaciones lógicas con valores constantes, mientras que el modo indirecto se utiliza para trabajar con valores almacenados en memoria. La elección del modo de direccionamiento adecuado depende de la tarea específica que se esté realizando.

Ensamblador Z80. Lógica (OR)

- La instrucción OR es una instrucción lógica que realiza una operación OR bit a bit entre el acumulador (registro A) y otro registro o valor. El resultado se almacena en el acumulador.
- La sintaxis es la siguiente:

OR registro/valor

- Siendo el registro uno de los siguientes: B, C, D, E, H, L, A
- O el valor inmediato de 8 bits en formato decimal o hexadecimal (prefijado por el símbolo #)

Ensamblador Z80. Lógica (XOR)

- La instrucción **OR establece las banderas (flags)** del registro F de la siguiente manera:
 - Z (Zero): se establece en 1 si el resultado de la operación es cero; en caso contrario, se establece en 0.
 - N (Add/Subtract): se establece en 0.
 - H (Half Carry): se establece en 0.
 - PV (Parity/Overflow): se establece en 1 si el número de bits a 1 en el resultado es par; en caso contrario, se establece en 0.
 - S (Sign): se establece en 1 si el bit más significativo (significante) del resultado es 1; en caso contrario, se establece en 0.
 - C (Carry): se establece en 0.

Ensamblador Z80. Lógica (XOR)

- La instrucción XOR sigue la sintaxis de AND y OR. Sus flags son:
 - Z (Zero flag): se establece en 1 si el resultado de la operación es cero, es decir, si todos los bits del acumulador y el operando son iguales a cero. En caso contrario, se establece en 0.
 - S (Sign flag): se establece en el bit más significativo del resultado de la operación.
 - P/V (Parity/Overflow flag): se establece en 1 si el número de bits a 1 en el resultado de la operación es par. En caso contrario, se establece en 0.
 - H (Half-carry flag): se establece en 1 si hubo un acarreo desde el bit 3 al bit 4 durante la operación. Si no, se establece en 0.
 - N (Add/Subtract flag): se establece en 0 para indicar que se ha realizado una operación de tipo XOR.
 - C (Carry flag): se establece en 0 ya que no se realiza ninguna operación de acarreo en la instrucción XOR.
- No modifica el contenido de los registros de propósito general (como B, C, D, E, H, L), sino que solo opera con el acumulador (A) y el operando especificado.

Ensamblador Z80. Labels

- Una etiqueta es un nombre simbólico asignado a una posición de memoria o una instrucción dentro del programa.
- Las etiquetas se utilizan para hacer referencia a direcciones de memoria específicas y facilitar la lectura y la comprensión del código.
 - Normalmente proporcionan un nombre significativo que identifica el propósito o la función de la ubicación de memoria. Por ejemplo:
 - En un programa que multiplica dos números, se podría utilizar una etiqueta "multiplicar" para referirse a la ubicación en el código donde se realiza la multiplicación, en lugar de referirse a la dirección de memoria específica.

Ensamblador Z80. Etiquetas.

Los usos/instrucciones más comunes de las etiquetas son:

- Marcar el **comienzo y fin de un programa**: **ORG** nn y **END** label;
- Etiquetar un valor: label **EQU** nn ; (este valor no podrá cambiar en el programa)
- Reserva de memoria:
 - label **DEFB** n, n,... ; guarda el valor n (8 bits) y sucesivos (00-FF) en la dirección de memoria correspondiente al puntero de dirección y sucesivas.
 - label **DEFW** nn, nn,... ; guarda el valor nn (16 bits) y sucesivas (0000-FFFF) en la dirección de memoria correspondiente al puntero de dirección y sucesivas.
 - label **DEFS** nn ; reserva nn (0000-FFFF) casillas de memoria a partir del puntero de dirección.
 - A “label” se le asigna la dirección de la primera casilla reservada.
 - El valor de las casillas no se modifica.

Ensamblador Z80. Etiquetas.

Los usos/instrucciones más comunes de las etiquetas son:

- Marcar el **comienzo y fin de un programa**: **ORG** nn y **END** label;
- Etiquetar un valor: label **EQU** nn ; (este valor no podrá cambiar en el programa)
- Reserva de memoria:
 - ...
 - label **DEFM** “cadena” ; almacena una cadena de caracteres ASCII en memoria a partir del puntero de dirección.
 - A “label” se le asigna la dirección de la casilla reservada en memoria con el primer carácter.
 - Cada carácter ASCII ocupa 16 bits, por lo tanto se necesitan registro dobles para su manejo

Ejercicio 5

Escribir un programa en ensamblador del Z80 que:

- Reserve memoria suficiente y almacene en ella los números 44H, 2AH, A3H y 71H.
- Reserve memoria suficiente y almacene en ella los números 10H, E2H, 6BH y 18H.
- Reserve memoria suficiente y almacene en ella la suma los números anteriores por pares (44H + 10H, 2AH + E2H, ...).

Solución

```
AppEntry    ld ix, numeros1
            ld iy, numeros2
            ld hl, resul
            ld b, 4

bucle       ld a, (ix)
            add a, (iy)
            ld (hl), a
            inc ix
            inc iy
            inc hl
            djnz bucle

fin          halt          ; Replace these lines
with your code

            jp fin          ;

numeros1     defb 44h, 2ah, 0a3h, 71h
numeros2     defb 10h, 0e2h, 6bh, 18h
resul        defs 4
```

```
            ld ix, oper1
            ld b, 4

bucle       ld a, (ix)
            add a, (ix + 4)
            ld (ix + 8), a
            inc ix
            djnz bucle

fin          halt
            jp fin

oper1        defb 44h, 2ah, 0a3h, 71h
oper2        defb 10h, 0e2h, 6bh, 18h
resul        defs 4
```

Ensamblador Z80. Desplazamiento y rotación

- Existen diversas instrucciones muy útiles para trabajar con números binarios y realizar operaciones de multiplicación o división por potencias de 2, así como para realizar operaciones de enmascaramiento y extracción de bits individuales. Los operandos son, todos los registros sencillos, y los registros indexados; (HL), (IX + d), (IY + d)
 - Las **instrucciones de desplazamiento** empiezan por S (shift), después se indica si el desplazamiento es a izquierda (L) o derecha (R) y finalmente si es desplazamiento lógico (L) o Aritmético (A)
 - Lista de instrucciones: SLA, SRA, SLL, SRL. Más nombre de Registro
 - **Rotación** empiezan por R (rotate) y puedes ser a la izquierda (L), a la derecha (R) y circulares (C) o no circulares:
 - Lista de instrucciones: RL, RR, RRC, RCL
 - Hay instrucciones en las que el **operando es el registro A**; RLCA, RLRC, RLA, RRA

Ensamblador Z80. Comparador

- La instrucción CP compara el contenido del acumulador con el del operando.
 - No cambia el valor del acumulador!
- Se encuentra en las siguientes formas:
 - CP R ; donde R = A, B, C, D, E, H, L
 - CP n ; donde n es un número de 00 a FF.
 - CP (HL)
 - CP (IX + d)
 - CP (IY + d)
- Si los valores son iguales, el flag Z = 1.
- Si A > operando, entonces Z = 0 y C = 0.
- Si A < operando, entonces Z = 0, y C = 1.

Ensamblador Z80. Saltos condicionales

- Dos de las bases de todo lenguaje de programación son las sentencias condicionales (IF) y los bucles (FOR/WHILE). En ensamblador del Z80, dichas funcionalidades se implementan a través de los saltos condicionales.
 - Un salto hace que la siguiente instrucción en ejecutar no sea la inmediatamente posterior a la que se está ejecutando, sino la referenciada por una etiqueta o la relativa a una posición de memoria concreta. Las instrucciones posibles de salto son:
 - JP ; usada para saltar a una instrucción situada en cualquier zona de memoria.
 - JR ; más ligera, usada para saltar a una distancia de [-127, +128] direcciones desde la dirección de la instrucción actual.
 - Se pueden encontrar como:
 - JP “label” ; JP (HL) ; JP (IX) ; JP (IY); JP (8050H)
 - JR “label”

Ensamblador Z80. Saltos condicionales

- Para establecer un salto condicional, se puede añadir una condición como primer operando de la instrucción. Las condiciones posibles son:

nemónico	significado	JP	JR
-	incondicional		
NZ	$Z = 0$		
Z	$Z = 1$		
NC	$C = 0$		
C	$C = 1$		
PO	$P/V = 0$		
PE	$P/V = 1$		
P	$S = 0$		
M	$S = 1$		

- Ejemplos:
 - JP Z, etiqueta1 ; salta a “etiqueta1” si $Z = 1$
 - JR NC, etiqueta2 ; salta a “etiqueta2” si $C = 0$

Ensamblador Z80. Bifurcaciones

- La instrucción DJNZ “label” es un caso especial, denominado decremento y bifurcación.
- Esta única instrucción ejecuta todo lo siguiente:
 - Decrementa el valor contenido en el registro B en 1.
 - Comprueba si B = 0.
 - En caso afirmativo, el programa continúa con la siguiente instrucción.
 - En caso contrario, salta a la etiqueta “label”.
- Es la manera más sencilla de crear un bucle.
- Siempre utiliza el registro B, por lo que hay que tener cuidado de inicializarlo correctamente sin pisar ningún valor importante.

Ejercicio 6

Escribir un programa en ensamblador del Z80 que:

- Almacene 10 números en memoria.
- Sume los 10 números y deje el resultado en otra posición de memoria previamente reservada.

Solución

```
org AppFirst          ; Start of application

AppEntry  ld b, 10
          ld hl, numeros
bucle     add a, (hl)
          inc hl
          djnz bucle

fin       halt
          jp fin      ;

numeros   defb 1,1,1,1,1,1,1,1,1,1
```

Ejercicio 7

Escribir un programa en ensamblador del Z80 que:

- Almacene un valor de una cifra en el registro C.
- Almacene un valor de una cifra en el registro D.
- Compare ambos valores y deje en el acumulador una cifra que indique el resultado de la comparación (1: $C > D$; 0: $C = D$; -1: $D > C$).

Solución

```
org AppFirst           ; Start of application

inicio  ld c, 2
        ld d, 2
        ld a, d
        cp c
        jp z, iguales   ; A=0, z=1 son iguales
        jp nc, dmayor   ; A=-1, z=0 son diferentes y c=0 a>operando, d>c
                        ; A=1, z=0 son diferentes y c=1 a<operando, d<c
        ld a, 1         ; aquí llegaría cuando d<c
        jp Inicio

iguales ld a,0
        jp Inicio

dmayor  ld a,-1
        jp Inicio
```


Ensamblador Z80. Rutinas

- Las rutinas son pequeños fragmentos de código incluidos en un programa ensamblador para realizar una función concreta.
- Las rutinas se deben etiquetar en su primera línea con una etiqueta aclaratoria de su función.
- Una rutina etiquetada “label” se puede invocar en cualquier parte del código, mediante la instrucción CALL “label”.
- La última instrucción de una rutina debe ser siempre RET. Cuando se ejecuta esta instrucción, el programa continúa por la instrucción siguiente a la llamada a la rutina (CALL).
- La llamada a la rutina puede ser condicional: CALL condición, “label” ; donde condición es cualquiera de las condiciones válidas para la instrucción de salto JP.
- Análogamente, la instrucción RET también puede ser condicional: RET condición ; donde condición es cualquiera de las condiciones válidas para la instrucción de salto JP.

Ejercicio 8

- Escribir un programa en ensamblador del Z80 que:
 - Almacene un valor de una cifra en el registro C.
 - Almacene un valor de una cifra en el registro D.
 - Compare ambos valores y deje en una dirección de memoria previamente reservada una cifra que indique el resultado de la comparación (1: $C > D$; 0: $C = D$; -1: $D > C$).
- Reescribir el programa para que efectúe 5 comparaciones consecutivas con 5 cifras diferentes, almacenando los resultados en 5 posiciones consecutivas de memoria.

Solución

```
org AppFirst
```

```
AppEntry    ld b, 5  
            ld ix, num1  
bucle       ld c, (ix)  
            ld d, (ix + 5)  
            call compara  
            ld (ix + 10), a  
            inc ix  
            djnz bucle  
            jp fin
```

```
compara     ld a, d  
            cp c  
            jp z, iguales  
            jp nc, mayor  
            ld a, -1  
            ret
```

```
iguales     ld a, 0  
            ret  
mayor       ld a, 1  
            ret
```

```
fin         halt  
            jp fin      ;
```

```
num1        defb 1,1,2,2,3  
num2        defb 3,3,2,2,1  
resul       defs 5
```

Ensamblador Z80. Pila

- La pila en Z80 es accesible por pares de registros de 1 byte o por registros de 2 bytes.
- PUSH RR es la instrucción para almacenar en la pila el contenido de RR.
- POP RR es la instrucción para extraer el valor contenido en la cima de la pila y almacenarlo en RR.
- RR puede ser: BC, DE, HL, AF, IX e IY.
- La pila se suele utilizar como método ágil de guardado temporal de los valores de un par de registros.
- También es útil para invertir el orden de una cadena.

Ejercicio 9

- Escribir un programa en ensamblador del Z80 que:
 - Almacene una cadena de caracteres ASCII de tamaño par.
 - Copie la cadena en otro lugar de la memoria, pero invertida. Se hará uso de la pila para llevar a cabo la operación.

Solución

```
org AppFirst

AppEntry    ld b, 4
            ld hl, cadena
bucle       ld a, (hl)
            push af
            inc hl
            djnz bucle
            ld b, 4
            ld hl, invertida
bucle2      pop af
            ld (hl), a
            inc hl
            djnz bucle2

fin         halt
            jp fin          ;

cadena      defm "hola"
invertida   defs 4
```

Ensamblador Z80. CPI

- A veces es necesario encontrar un cierto valor dentro de una cadena de valores.
- La instrucción **CPI** automatiza este proceso, comparando el byte almacenado en el acumulador con el byte almacenado en la dirección de memoria contenida en HL.
- Al ejecutarse CPI, se realizan tres operaciones:
 - $A - (HL)$; en A se almacena el **octeto buscado**
 - $HL + 1$; HL sirve de **puntero** para recorrer todas las posiciones de la cadena
 - $BC - 1$; BC sirve como **contador** (byte counter) del número de bytes comparados
- El flag Z = 1 si $A = (HL)$. El flag P/V = 1 mientras $BC - 1$ no sea 0.
- La instrucción **CPIR** repite este proceso de manera automática hasta que $BC = 0$.
- BC debe inicializarse con el tamaño de la cadena. Si $BC = 0$ antes de ejecutar la instrucción, el bucle se repite 64k veces o hasta que $A = (HL)$.

Ejercicio

- Escribir un programa en ensamblador del Z80 que:
 - Almacene una secuencia de 5 números consecutivamente.
 - Busque un número concreto dentro de dicha secuencia.
 - Deje en A una cifra indicando si lo ha encontrado o si no (1: encontrado; 0: no encontrado).

Solución

```
org AppFirst

AppEntry    ld bc, 5
            ld hl, cadena
            ld a, 4
            cpir
            jp z, encontrado
            ld a, 0
            jp fin
encontrado  ld a, 1

fin          halt
            jp fin

cadena      defb 1,2,3,4,5
```